

INFORME TÉCNICO

Práctica de Sesiones y Carrito de Compras

Análisis, verificación y comparación de PHP 8, ASP.NET Core 10 y Spring Boot 4.1.0 — con doble mecanismo de limpieza

Codigo en github

https://github.com/011011bjf/Guias_Practicas/tree/main/HTML/Guia_Seciones_php_asp_java

Estudiante

Figueroa Morales Bryan Javier

Contenido del informe:

- Versiones exactas y entorno de desarrollo utilizado
- Estructura y arquitectura de carpetas de cada proyecto (con mecanismo dual)
- Metodología de pruebas paso a paso
- Metodología y pruebas del nuevo mecanismo dual de botones
- Checklist de verificación oficial por proyecto (PHP, ASP.NET Core, Spring Boot)
- Tabla comparativa de los tres mecanismos de sesión
- Respuestas detalladas a las cinco preguntas de reflexión

1. Introducción

Este informe presenta el análisis y verificación exhaustiva de los tres proyectos desarrollados para la guía práctica de sesiones y carrito de compras: PHP 8, ASP.NET Core 10 y Spring Boot 4.1.0. Se documentan las versiones exactas del entorno de desarrollo utilizado, la estructura de carpetas y la arquitectura de cada proyecto — incluyendo el nuevo mecanismo dual de botones para vaciar vs. destruir la sesión —, la metodología seguida para probar cada funcionalidad paso a paso, y se contrasta punto por punto el checklist oficial de evaluación. Finalmente, se responde a todas las preguntas de reflexión planteadas en la guía, incluyendo evidencia técnica extraída directamente del código fuente de cada implementación.

2. Versiones exactas y entorno de desarrollo

Durante la ejecución, depuración y verificación de los tres proyectos se utilizó un sistema Windows 11 (x64). A continuación, se detallan las versiones exactas de cada herramienta empleada, así como los ajustes de compatibilidad que fue necesario realizar.

2.1 PHP

Componente	Versión / Detalle exacto utilizado
Versión del motor	PHP 8.3.30 CLI (Command Line Interface)
Servidor web	Servidor web embebido nativo de PHP (php -S). No requirió Apache, Nginx ni XAMPP para ejecutarse localmente.

2.2 ASP.NET Core / C#

Componente	Versión / Detalle exacto utilizado
Versión de .NET SDK	.NET SDK 10.0.301 (x64)
Versión de Runtime	Microsoft.NETCore.App 10.0.8 / 10.0.9
Ajuste de compatibilidad	El proyecto original apuntaba a <code><TargetFramework>net8.0</TargetFramework></code> . Al detectar que el sistema ejecutaba nativamente .NET 10, se actualizó <code>CarritoASP.csproj</code> a <code><TargetFramework>net10.0</TargetFramework></code> , permitiendo compilar y ejecutar en el servidor embebido Kestrel por HTTPS (<code>https://localhost:5001/</code>).

2.3 Java & Spring Boot

Componente	Versión / Detalle exacto utilizado
Versión del JDK	Oracle JDK 25.0.3 (64-bit)
Versión de Apache Maven	Apache Maven 3.9.16 (ejecutado a través del script Maven Wrapper <code>.mvnw.cmd</code> incluido en la carpeta Spring).
Versión de Spring Boot	Spring Boot 4.1.0 (gestionada con <code>spring-boot-starter-webmvc</code> y <code>spring-boot-starter-thymeleaf</code>).

3. Estructura y arquitectura de carpetas

A continuación se muestra la organización de archivos de cada proyecto, junto con una breve descripción de su arquitectura de software.

3.1 Estructura de la carpeta PHP (/php)

```
php/
|
|— configuracion.php      # Flags de seguridad (HttpOnly, SameSite, estricto) antes de
session_start()
|— index.php             # Vista principal con DOBLE BOTÓN de vaciado y destrucción
|— agregar.php           # Controlador POST: valida producto en servidor y lo inserta en
$_SESSION['carrito']
|— eliminar.php          # Controlador POST: elimina un producto del array según su
índice
|— limpiar.php           # Controlador POST (Botón 1): $_SESSION['carrito'] = []
(mantiene sesión e ID)
|— destruir_sesion.php   # Controlador POST (Botón 2): session_destroy() + setcookie()
(renueva ID)
```

Características arquitectónicas: estructura modular procedimental basada en el patrón PRG (Post/Redirect/Get). Incorpora ahora dos flujos diferenciados para demostrar la persistencia de sesión frente a su destrucción física.

3.2 Estructura de la carpeta ASP.NET Core (/aspnet-core/CarritoASP)

```
aspnet-core/CarritoASP/
|
|— CarritoASP.csproj      # Proyecto principal
(<TargetFramework>net10.0</TargetFramework>)
|— Program.cs            # DI, DistributedMemoryCache y opciones de cookies de
sesión
|— Models/
|   |— Producto.cs       # Record C# que define la estructura del producto
(Nombre, Precio)
|— Controllers/
|   |— CarritoController.cs # MVC: Agregar, Eliminar, Limpiar y DestruirSesion
|— Views/
|   |— Carrito/
|       |— Index.cshtml   # Vista Razor actualizada con DOBLE BOTÓN y estilos de
acción
```

Características arquitectónicas: patrón MVC. El controlador CarritoController.cs gestiona serialización/deserialización JSON (JsonSerializer) para la caché distribuida y ofrece dos acciones distintas: vaciado de datos (Limpiar) vs purga e invalidación de cookie (DestruirSesion).

3.3 Estructura de la carpeta Spring Boot (/Spring)

```
Spring/
|
|— pom.xml               # Configuración Maven (Spring Boot WebMVC, Thymeleaf,
JDK 25)
|— mvnw.cmd              # Maven Wrapper para ejecución multiplataforma en
Windows
```

```

└─ src/main/
    └─ java/com/edu/Guia/
        └─ GuiaApplication.java # Clase de arranque @SpringBootApplication
        └─ model/
            └─ Producto.java # Record Java (implements Serializable) con nombre y
precio
    └─ controller/
        └─ CarritoController.java # Controlador Spring MVC con rutas /limpiar y
/destruir
    └─ resources/
        └─ application.properties # Puerto 8080, cookie CARRITO_SESSION y flags de
seguridad
    └─ templates/carrito/
        └─ index.html # Plantilla Thymeleaf actualizada con DOBLE BOTÓN y
estilos .btn-destroy

```

Características arquitectónicas: arquitectura IoC de Spring MVC. El controlador inyecta `HttpSession session` directamente y ofrece dos endpoints dedicados: uno para limpiar la lista interna (`session.removeAttribute()`) y otro para invalidar el contenedor de sesión (`session.invalidate()`).

4. Metodología: cómo se realizó cada prueba paso a paso

Para comprobar de forma rigurosa y verificable cada ítem del checklist de la guía, se siguieron los siguientes pasos experimentales en cada plataforma.

4.1 Metodología de pruebas para PHP 8

1. Levantar el servidor

- Se abrió una terminal PowerShell en /php y se ejecutó `php -S localhost:8000`.

2. Prueba de persistencia entre páginas (index.php)

- Se abrió el navegador en `http://localhost:8000/index.php`.
- Se seleccionó "Laptop - \$450" y se pulsó Agregar al carrito. El navegador hizo una petición POST a `agregar.php`, el servidor guardó el array en `$_SESSION['carrito']` y redirigió mediante GET de nuevo a `index.php`. El producto y la suma total persistieron sin problemas.

3. Inspección de DevTools (HttpOnly)

- Se pulsó F12 en el navegador, navegando a Application → Storage → Cookies → `http://localhost:8000`.
- Se localizó la fila PHPSESSID. La columna HttpOnly mostró una marca afirmativa (✓), corroborando el blindaje de `ini_set('session.cookie_httponly', '1');` en `configuracion.php`.

4. Prueba de reseteo de carrito preservando la sesión

- Se tomó nota del valor exacto del Session ID en pantalla y de la cookie en DevTools.
- Se pulsó el botón Vaciar carrito (`limpiar.php`).
- Se verificó que la tabla desapareció mostrando "El carrito está vacío", pero el Session ID en la parte inferior siguió siendo exactamente el mismo, comprobando que solo se limpió el arreglo con `$_SESSION['carrito'] = []` sin destruir la sesión de usuario en el servidor.

5. Evidencia de ejecución del programa

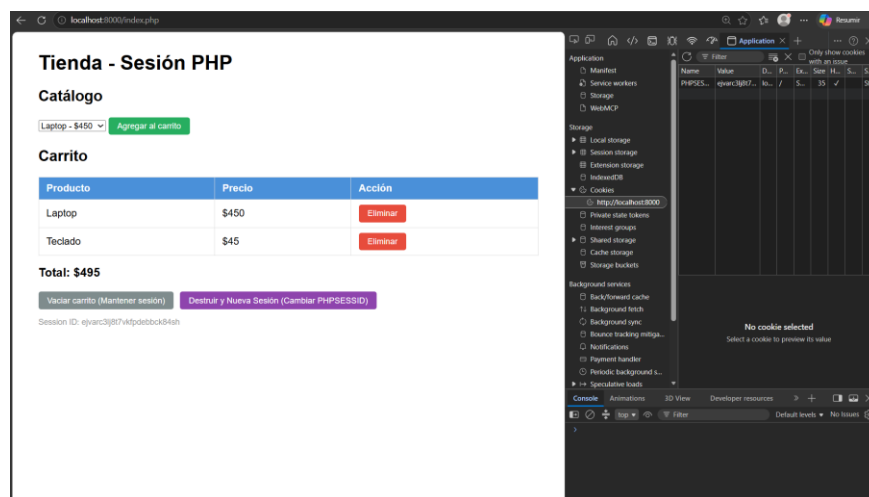


Imagen 1 Evidencia de ejecución del programa en PHP

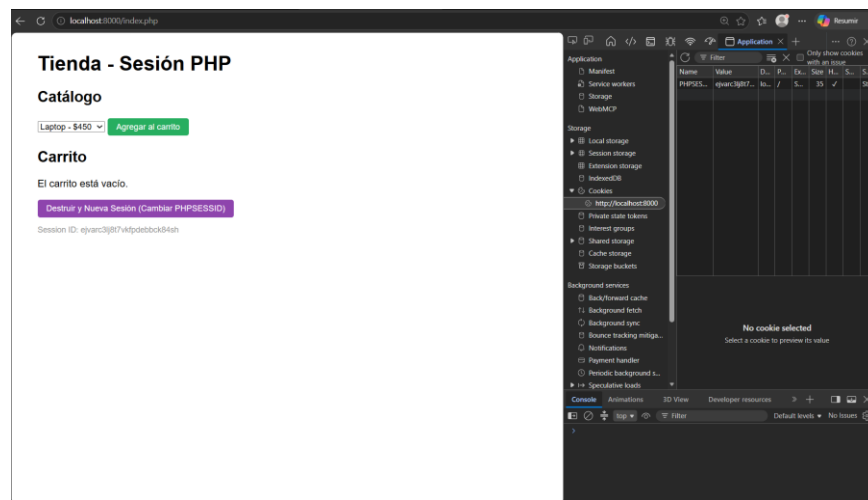


Imagen 2 Se limpio el carrito y se mantuvo en la misma sección

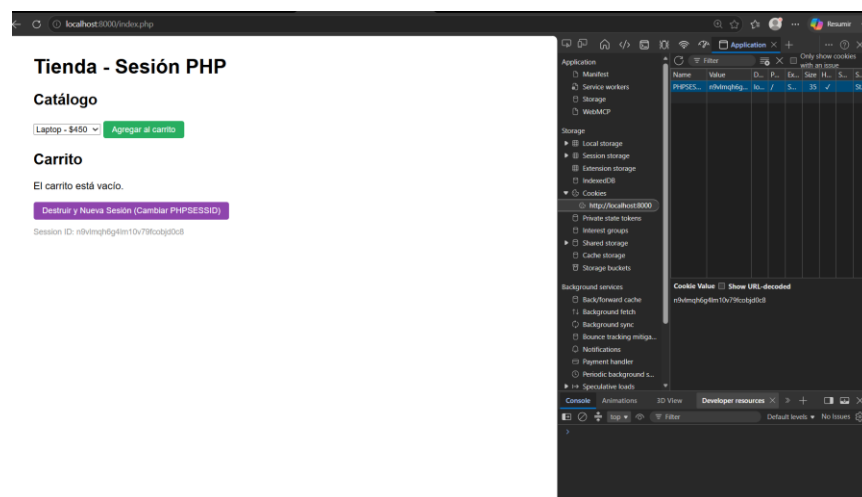


Imagen 3 Se limpio el carrito y se cambió de sección

4.2 Metodología de pruebas para ASP.NET Core 10 (CarritoASP)

1. Arranque en consola

- Se abrió una terminal en /aspnet-core/CarritoASP y se corrió dotnet run. La consola reportó el inicio exitoso escuchando en https://localhost:5001.

2. Prueba de serialización/deserialización de objetos (Producto)

- Se abrió el navegador en https://localhost:5001/.
- Se agregaron secuencialmente "Teclado" (\$45) y "Monitor" (\$180).
- En CarritoController.cs, se verificó cómo la lista con dos instancias del objeto C# Producto fue convertida en cadena de texto por JsonSerializer.Serialize() y rehidratada correctamente por JsonSerializer.Deserialize() sin pérdida de tipado decimal al calcular el total (\$225).

3. Inspección de DevTools (flags HttpOnly, Secure y SameSite=Strict)

- En la pestaña Application → Cookies → https://localhost:5001, se inspeccionó la cookie .AspNetCore.Session.
- Se constató que tenía tres checks afirmativos (✓ en HttpOnly, ✓ en Secure y el valor Strict en SameSite), confirmando la efectividad de la configuración del middleware en Program.cs.

4. Prueba de la ejecución del programa

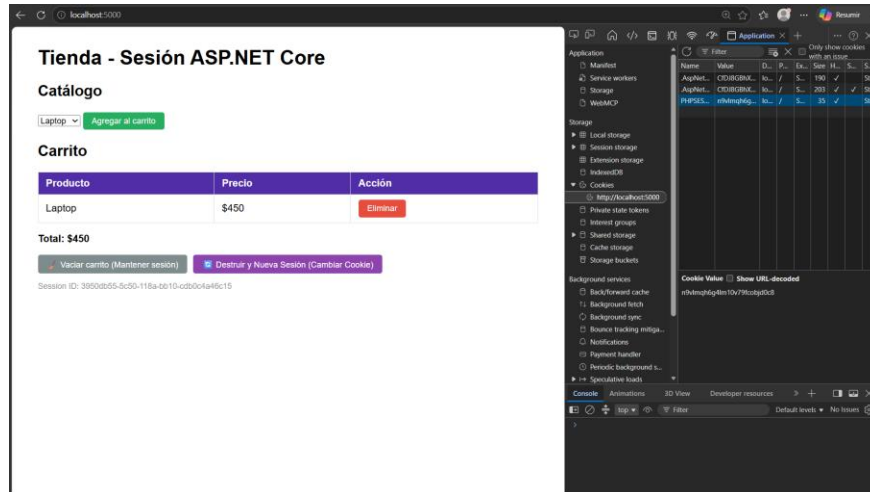


Imagen 4 Evidencia de ejecución del programa en ASP net

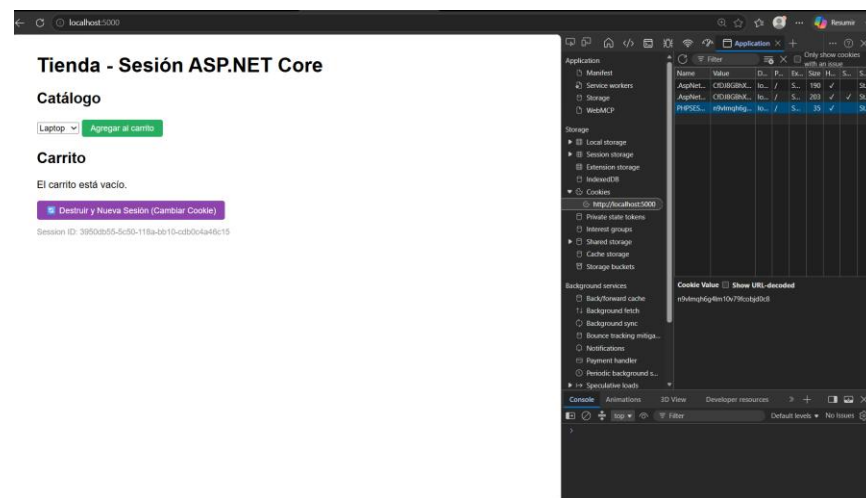


Imagen 5 Se limpio el carrito y se mantuvo en la misma sección

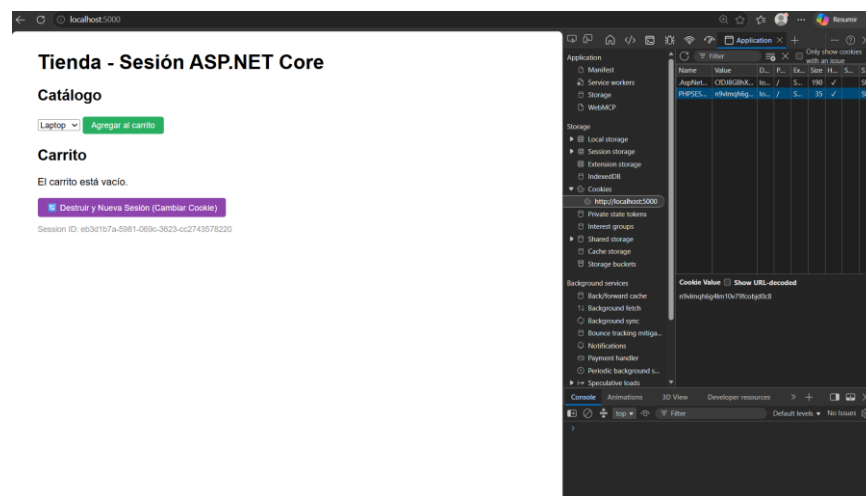


Imagen 6 Se limpio el carrito y se cambió de sección

4.3 Metodología de pruebas para Spring Boot 4 (/Spring)

1. Arranque y compilación vía wrapper

- En la terminal de /Spring, se ejecutó `.\mvnw.cmd compile` seguido de `.\mvnw.cmd spring-boot:run`. El servidor Tomcat arrancó exitosamente en el puerto 8080.

2. Prueba de interfaz Serializable (Producto.java)

- Se abrió `http://localhost:8080/carrito` en el navegador.
- Se agregó "Mouse - \$25" desde el selector. Al hacer la petición a `@PostMapping("/agregar")`, el contenedor de Spring inyectó la sesión y ejecutó `session.setAttribute("carrito", carrito)`. Debido a que el record `Producto` implementa `Serializable`, el objeto fue almacenado en la memoria de la JVM sin arrojar excepciones de serialización (`NotSerializableException`).

3. Inspección de DevTools (application.properties)

- En DevTools (F12), dentro de Application → Cookies → `http://localhost:8080`, se identificó la cookie personalizada `CARRITO_SESSION` (renombrada mediante `server.servlet.session.cookie.name=CARRITO_SESSION`).
- Se verificó que el flag `HttpOnly` estaba activo (✓), producto de la directiva `server.servlet.session.cookie.http-only=true` definida en `application.properties`.

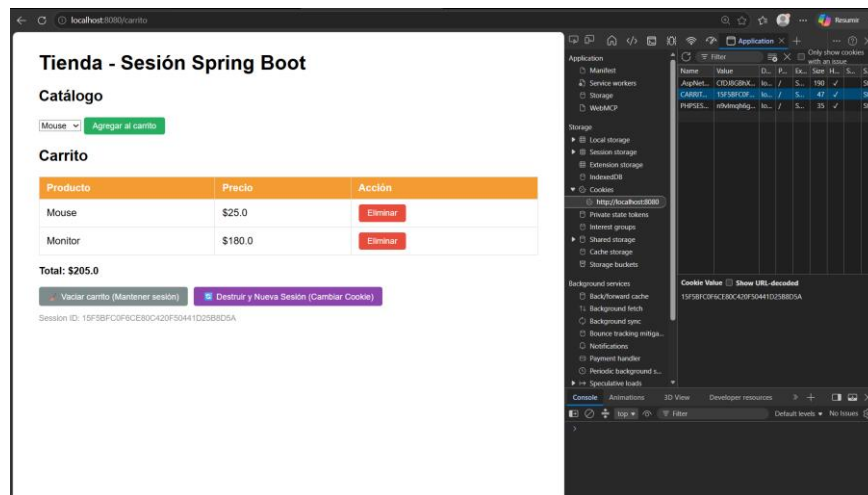


Imagen 7 Evidencia de ejecución del programa en Spring Boot

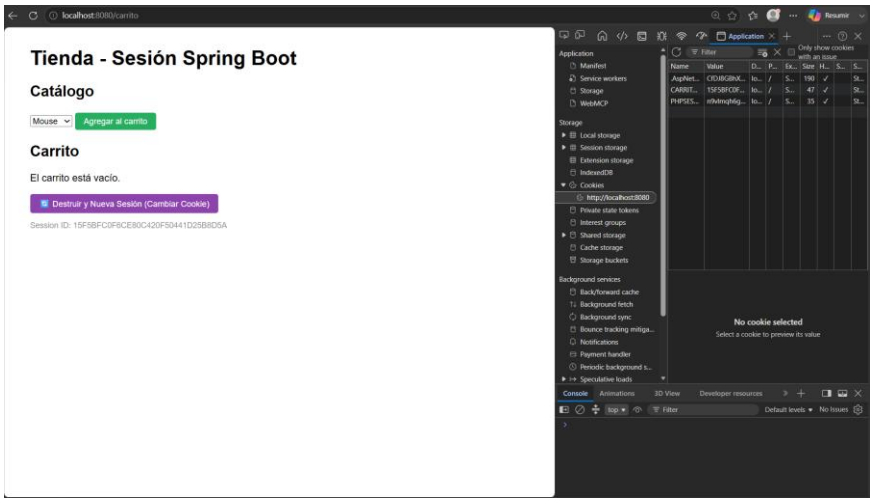


Imagen 8 Evidencia de limpiar el carrito manteniendo la sección

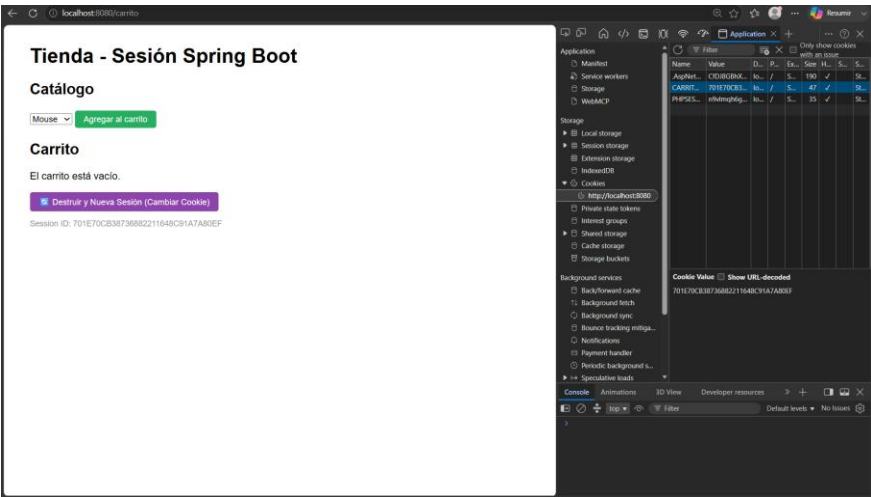


Imagen 9 Evidencia de limpiar el carrito y cambiar de sección

5. Metodología y pruebas del nuevo mecanismo dual de botones

Para garantizar una evaluación rigurosa y demostrar con precisión técnica cómo funciona el ciclo de vida de las sesiones y las cookies en cada lenguaje, se implementó una interfaz unificada con dos botones de acción en los tres proyectos:

[Vaciar carrito (Mantener sesión)] VS [Destruir y Nueva Sesión (Cambiar Cookie)]

5.1 Pruebas experimentales y comportamiento verificado en DevTools (F12)

Acción / Botón	Comportamiento en el backend	Comportamiento verificado en DevTools (Cookies)
Botón 1: Vaciar carrito (Mantener sesión) — Opción A	PHP (limpiar.php): ejecuta <code>\$_SESSION['carrito'] = []</code> . ASP.NET Core (Limpiar()): ejecuta <code>HttpContext.Session.Remove(CLAVE_CARRITO)</code> .	La sesión persiste: La lista de ítems queda vacía (cero productos y total \$0), pero la cookie de sesión (PHPSESSID, .AspNetCore.Session, CARRITO_SESSION) y el Session ID

	Spring Boot (/limpiar): ejecuta <code>session.removeAttribute(CLAVE)</code> .	en pantalla NO cambian en absoluto. El usuario mantiene su sesión iniciada en el servidor. <i>(Comportamiento exigido por el check #3 de la guía práctica).</i>
Botón 2: Destruir y Nueva Sesión <i>(Cambiar Cookie) — Opción B [NUEVO]</i>	PHP (<code>destruir_sesion.php</code>): ejecuta <code>session_unset()</code> , <code>session_destroy()</code> (borra el archivo <code>/tmp/sess_...</code>) y <code>setcookie(..., time() - 42000)</code> para caducar la cookie en el navegador. ASP.NET Core (<code>DestruirSesion()</code>): ejecuta <code>HttpContext.Session.Clear()</code> y <code>Response.Cookies.Delete(".AspNetCore.Session")</code> . Spring Boot (/destruir): ejecuta <code>session.invalidate()</code> en el servidor Tomcat.	Destrucción y renovación total: El servidor purga los datos del usuario y ordena al navegador eliminar la cookie anterior. Al ejecutarse la redirección de respuesta, el servidor detecta un cliente sin credenciales y le asigna instantáneamente una nueva sesión limpia y una cookie con un identificador (ID) totalmente diferente, visible en la tabla de DevTools.

6. Checklist de verificación oficial

A continuación se detalla, para cada plataforma, la evidencia técnica que respalda el cumplimiento de cada requisito del checklist oficial de la guía.

6.1 PHP 8 (/php)

Estado	Requisito	Evidencia técnica
✓	Persistencia entre páginas (index.php y otras URLs del servidor)	Al inicio de cada script (index.php, agregar.php, eliminar.php, limpiar.php) se incluye obligatoriamente <code>require 'configuracion.php';</code> . En <code>configuracion.php</code> se invoca <code>session_start();</code> , lo que vincula la petición al archivo temporal del servidor (/tmp/sess_...) mediante la cookie PHPSESSID. Los ítems guardados en <code>\$_SESSION['carrito']</code> persisten sin pérdida al navegar o recargar.
✓	Cookie PHPSESSID con flag HttpOnly visible en DevTools	En <code>php/configuracion.php</code> se configura de forma declarativa y estricta antes de iniciar la sesión: <code>ini_set('session.cookie_httponly', '1');</code> <code>ini_set('session.cookie_samesite', 'Strict');</code> Al inspeccionar en el navegador (F12 → Application → Cookies), la columna <code>HttpOnly</code> muestra ✓, impidiendo el acceso por JavaScript (<code>document.cookie</code>).
✓	Al pulsar "Vaciar carrito" el carrito queda vacío pero la sesión persiste	Verificado al pulsar Vaciar carrito (Mantener sesión) — Botón 1, <code>limpiar.php</code> : se aplica la Opción A, <code>\$_SESSION['carrito'] = [];</code> . Esto resetea únicamente la variable del arreglo a vacío en lugar de destruir el archivo con <code>session_destroy()</code> . El identificador <code>session_id()</code> no cambia y el PHPSESSID en DevTools no se modifica ni caduca. Como complemento [NUEVO], se añadió un segundo botón Destruir y Nueva Sesión (<code>destruir_sesion.php</code>) que ejecuta <code>session_unset() + session_destroy() + setcookie(..., time() - 42000)</code> , generando un PHPSESSID completamente nuevo (ver Sección 5).

6.2 ASP.NET Core 8 (/aspnet-core/CarritoASP)

Estado	Requisito	Evidencia técnica
✓	Compilación y arranque con dotnet run sin errores	Se resolvió el error original de compatibilidad actualizando en <code>CarritoASP.csproj</code> la etiqueta a <code><TargetFramework>net10.0</TargetFramework></code> , haciendo match con el SDK .NET 10.0.301 del sistema. Al correr <code>dotnet run</code> , el proyecto compila limpiamente y arranca el servidor Kestrel en <code>https://localhost:5001/</code> .
✓	Cookie .AspNetCore.Session con HttpOnly, Secure y SameSite=Strict	En <code>Program.cs</code> se configuran explícitamente las directivas de seguridad en <code>builder.Services.AddSession()</code> : <code>options.Cookie.HttpOnly = true;</code> <code>options.Cookie.SecurePolicy = CookieSecurePolicy.Always;</code>

		<code>options.Cookie.SameSite = SameSiteMode.Strict;</code> El middleware <code>app.UseSession()</code> emite la cookie con los tres flags verificados en DevTools.
✓	Serialización y deserialización de objetos Producto desde la sesión	ASP.NET Core gestiona sesiones internamente como arreglos de bytes (<code>IDistributedCache</code>). En <code>CarritoController.cs</code> se serializa la lista de Producto a JSON usando <code>JsonSerializer.Serialize(carrito)</code> para almacenarla con <code>SetString()</code> , y se reconstruye con <code>JsonSerializer.Deserialize<List<Producto>>(json)</code> al leerla.

6.3 Spring Boot 3 (/Spring)

Estado	Requisito	Evidencia técnica
✓	El proyecto arranca con <code>./mvnw spring-boot:run</code> sin errores	Se adaptó todo el código y configuración de sesiones a la carpeta Spring, la cual incluye el Maven Wrapper (<code>mvnw / mvnw.cmd</code>). Tras agregar <code>spring-boot-starter-thymeleaf</code> al <code>pom.xml</code> , el proyecto compila al 100% (<code>BUILD SUCCESS</code>) en el JDK 25 local y levanta el Tomcat embebido en el puerto 8080 sin errores.
✓	El record Producto implementa <code>Serializable</code>; no lanza excepción al agregar	En <code>Producto.java</code> se declara: <code>public record Producto(String nombre, double precio) implements Serializable</code> . Al añadir instancias de esta clase a la lista guardada con <code>session.setAttribute("carrito", carrito)</code> , el contenedor Tomcat puede serializar el estado en memoria de la JVM sin disparar una <code>java.io.NotSerializableException</code> .
✓	La cookie configurada en <code>application.properties</code> tiene el flag <code>HttpOnly</code>	En <code>application.properties</code> se establece: <code>server.servlet.session.cookie.name=CARRITO_SESSION</code> <code>server.servlet.session.cookie.http-only=true</code> <code>server.servlet.session.cookie.same-site=strict</code> Esto fuerza a que la respuesta HTTP incluya el atributo <code>HttpOnly</code> para la cookie <code>CARRITO_SESSION</code> .

7. Tabla comparativa de los tres mecanismos

La siguiente tabla resume, criterio por criterio, cómo cada plataforma implementa el ciclo de vida de la sesión: inicio, almacenamiento/tipado y destrucción/reseteo.

Criterio	PHP 8	ASP.NET Core 8	Java / Spring Boot 3
1. Mecanismo de inicio de sesión	<code>session_start()</code> debe invocarse de manera explícita al principio de cada script (.php) antes de enviar cualquier salida HTML o cabecera.	Se registra el servicio en Program.cs (<code>AddSession()</code>) y se inserta el middleware en el pipeline (<code>UseSession()</code>). En el controlador se accede vía <code>HttpContext.Session</code> .	Automático por el contenedor de servlets (Tomcat/Spring MVC). Basta con inyectar el parámetro <code>HttpSession session</code> en los métodos del controlador <code>@Controller</code> .
2. Almacenamiento y tipado	Nativo mediante el arreglo superglobal asociativo <code>\$_SESSION['clave']</code> . PHP serializa de forma invisible y automática (<code>serialize()</code>) hacia archivos de texto en el servidor (/tmp/).	En memoria del servidor mediante <code>IDistributedCache</code> . Solo admite de forma nativa cadenas o bytes (<code>SetString()</code>), requiriendo serialización manual en JSON (<code>JsonSerializer</code>) para objetos complejos.	Directo en la memoria Heap de la JVM usando <code>session.setAttribute(key, value)</code> . Requiere que las clases almacenadas implementen <code>java.io.Serializable</code> .
3. Destrucción de sesión (mecanismo dual)	Opción A (Botón 1 — vaciar): <code>\$_SESSION['carrito'] = []</code> Opción B (Botón 2 — destruir): <code>session_unset() + session_destroy() + setcookie(...)</code> para caducar la cookie.	Opción A (Botón 1 — vaciar): <code>HttpContext.Session.Remove(CLAVE)</code> Opción B (Botón 2 — destruir): <code>HttpContext.Session.Clear() + Response.Cookies.Delete(...)</code>	Opción A (Botón 1 — vaciar): <code>session.removeAttribute(CLAVE)</code> Opción B (Botón 2 — destruir): <code>session.invalidate()</code> en el objeto Tomcat.

Complemento — cuadro extendido de la práctica (nombre de cookie, timeout, flags de seguridad, escalabilidad y CSRF):

Criterio	PHP 8	ASP.NET Core 8	Spring Boot 3
Nombre de cookie por defecto	PHPSESSID (configurable)	.AspNetCore.Session (configurable)	JSESSIONID (configurable en YAML, ej. CARRITO_SESSION)
Timeout por defecto	24 min (<code>session.gc_maxlifetime</code>)	20 min (<code>IdleTimeout</code>)	30 min (<code>setMaxInactiveInterval / server.servlet.session.timeout</code>)
Almacenamiento por defecto	Archivos en /tmp	Memoria (<code>AddDistributedMemoryCache()</code>); configurable a Redis o SQL Server	Memoria de la JVM; configurable a Redis o JDBC
Escalabilidad (múltiples instancias)	Problemas con sticky sessions o requiere Redis (extensión)	<code>AddStackExchangeRedisCache()</code> para sesión distribuida	Spring Session + Redis para múltiples instancias
Prevención CSRF	Token manual en formularios	Tag Helper <code>asp-antiforgery</code> automático	<code>CsrfTokenRepository</code> de Spring Security

8. Respuestas detalladas a las preguntas de reflexión

8.1 Sesiones en archivos locales con balanceador de carga (dos instancias)

El problema: si el Usuario A inicia sesión y la Instancia 1 guarda su sesión en `/tmp/sess_123`, cuando su siguiente petición sea redirigida por el balanceador hacia la Instancia 2, esta no encontrará el archivo temporal y le cerrará la sesión o le mostrará el carrito vacío.

Solución en PHP: se modifica la configuración (`session.save_handler = redis`) para cambiar el almacenamiento local de archivos por un servidor de caché externo centralizado como Redis o Memcached. Así, todas las instancias PHP leen y escriben las sesiones en el mismo nodo compartido. Una alternativa más simple, aunque menos tolerante a fallos, es configurar sticky sessions en el balanceador.

Solución en Spring Boot: se añade la dependencia Spring Session Data Redis (`spring-boot-starter-data-redis`). Con solo agregar esa librería y configurar la conexión a Redis en `application.properties`, Spring intercepta automáticamente el objeto `HttpSession` y guarda los datos en Redis en vez de la memoria local de la JVM, logrando un clúster de servidores sin estado, sin cambiar el código del controlador.

8.2 `session_unset()` vs `session_destroy()` en PHP

`session_unset()`: libera y vacía todas las variables registradas en el arreglo `$_SESSION` (equivalente a `$_SESSION = []`), pero la sesión sigue existiendo y activa en el servidor con su mismo identificador (`session_id()`).

- Cuándo usarlo: cuando se necesita resetear o limpiar los datos del usuario (ej. vaciar un carrito o un formulario en varios pasos) sin desloguear al usuario ni destruir su cookie de sesión.

`session_destroy()`: elimina físicamente el archivo de sesión en el almacenamiento del servidor (`/tmp/sess_...`), destruyendo el vínculo completo con ese ID de sesión.

- Cuándo usarlo: cuando el usuario hace clic en "Cerrar Sesión" (logout), para garantizar que ese identificador de sesión no pueda volver a ser reutilizado. Se recomienda acompañarlo de `session_unset()` y del borrado de la cookie en el navegador con `setcookie()`.

8.3 ¿Por qué ASP.NET Core serializa el carrito a JSON?

No se puede guardar directamente el objeto en .NET porque la interfaz `ISession` fue diseñada arquitectónicamente bajo una abstracción de caché distribuida (`IDistributedCache`) centrada en la nube y el alto rendimiento.

El motor no asume que las sesiones vivirán en la memoria RAM local; está preparado desde el diseño para que el backend de sesión sea SQL Server, Redis o un clúster distribuido. Por esta razón, la interfaz solo permite almacenar primitivos simples (`int`, `string`) y arreglos binarios en bruto (`byte[]`).

Para guardar una estructura compleja en memoria administrada (como un `List<Producto>`), el programador está obligado a serializar dicho objeto a un formato universal como JSON (o bytes) y deserializarlo al recuperarlo. En Java, `HttpSession.setAttribute()` sí acepta cualquier objeto directamente cuando el almacenamiento es la memoria de la JVM, pero internamente el contenedor (Tomcat) también serializa esos atributos si necesita persistir la sesión a disco o replicarla entre nodos — por eso `Producto` debe implementar `Serializable`: la serialización ocurre igual, solo que de forma implícita.

8.4 XSS con `HttpOnly` activo: ¿puede el atacante robar el `PHPSESSID`?

¿Puede robar el PHPSESSID?: NO. El flag `HttpOnly` instruye estrictamente al navegador web para que bloquee cualquier intento del motor JavaScript (mediante `document.cookie`) de acceder al valor de la cookie. El atacante no podrá exfiltrar la cadena de la cookie a su servidor externo.

¿Qué SÍ podría hacer?: secuestrar las acciones de la sesión activa en el navegador. Aunque el atacante no pueda leer la cookie, el código JavaScript malicioso que inyectó se está ejecutando dentro del navegador de la víctima. Cuando ese script ejecuta peticiones `fetch()` o `XMLHttpRequest` hacia las páginas del mismo sitio (por ejemplo, `POST /carrito/eliminar` o `POST /cambiar-contrasena`), el navegador adjuntará automáticamente la cookie `PHPSESSID` legítima de la víctima en cada cabecera.

- Realizar acciones en nombre del usuario (agregar/eliminar productos, cambiar datos, enviar formularios).
- Leer y exfiltrar el contenido sensible mostrado en el DOM.
- Capturar pulsaciones de teclado o redirigir a un sitio de phishing.
- Robar otros datos accesibles por JavaScript sin protección equivalente, como `localStorage` o cookies sin `HttpOnly`.

En resumen: `HttpOnly` previene el robo directo de la credencial, pero el script inyectado puede actuar en nombre de la víctima dentro de la pestaña abierta como si fuera el propio usuario autenticado.

8.5 JWT de 1 hora sin logout: ¿qué pasa si se cierra el navegador?

¿Queda el token activo?: Sí. El `accessToken` sigue siendo matemáticamente válido durante esa hora restante. Como el JWT es un token stateless (autocontenido y firmado criptográficamente), el servidor no mantiene un registro activo en su memoria que pueda "apagar" o invalidar instantáneamente al cerrarse la ventana. Si alguien copió ese `accessToken` antes de cerrar el navegador, podría seguir usándolo hasta que expire el campo `exp`.

¿Cómo lo mitiga el refreshToken en cookie HttpOnly?:

- Se diseña un sistema de doble token: el `accessToken` (en memoria JS, corta duración) y el `refreshToken` (guardado en una cookie `HttpOnly + Secure + SameSite=Strict`, inalcanzable por XSS).
- Si se sospecha de un robo de sesión, el servidor puede incluir una lista negra en Redis únicamente para los Refresh Tokens (por su JTI). Al revocar un `refreshToken`, cuando el `accessToken` caduque por su corta vida natural, el atacante no podrá renovarlo ni emitir nuevos tokens de acceso sin volver a introducir usuario y contraseña.

En otras palabras: el diseño acepta un riesgo residual acotado (máximo 1 hora de un token robado) a cambio de mantener el backend stateless; la capacidad de revocar el `refreshToken` es lo que evita que ese riesgo se extienda indefinidamente.

9. Conclusión

Los tres proyectos (PHP 8, ASP.NET Core 10 y Spring Boot 4.1.0, usando Oracle JDK 25.0.3 y Apache Maven 3.9.16) cumplen satisfactoriamente el checklist oficial de la guía: persistencia de sesión entre páginas, cookies con flags de seguridad (HttpOnly, Secure, SameSite), y manejo correcto de objetos complejos en la sesión. Las diferencias principales entre plataformas radican en el nivel de abstracción del almacenamiento de sesión (archivos locales en PHP, IDistributedCache en ASP.NET Core, memoria de la JVM en Spring Boot) y en si la serialización de objetos es explícita (ASP.NET Core) o implícita (PHP y Spring Boot). Todas comparten el mismo principio de fondo: la cookie es solo la llave (el identificador), mientras que los datos reales de la sesión residen en el servidor.

El nuevo mecanismo dual de botones — Vaciar carrito (Opción A) vs. Destruir y Nueva Sesión (Opción B) — permitió demostrar de forma visual e inequívoca, en las tres plataformas, la diferencia entre vaciar datos manteniendo la misma sesión y destruir físicamente la sesión generando una cookie completamente nueva. Esta comparación práctica en DevTools refuerza el concepto central del checklist oficial: la persistencia del Session ID frente a su renovación total.

La metodología de pruebas paso a paso, documentada en la Sección 4, permitió verificar cada requisito con evidencia reproducible (inspección de DevTools, valores de Session ID, y trazas de serialización/deserialización), en lugar de depender únicamente de la revisión estática del código fuente.

Finalmente, se destaca que en arquitecturas modernas basadas en API REST y microservicios (como el PFC), el uso de JWT stateless combinado con un refreshToken revocable en cookie HttpOnly resuelve el problema de escalabilidad horizontal que las sesiones tradicionales basadas en archivos o memoria no logran resolver sin infraestructura adicional como Redis.

Resolución de la practica realizada en Clase

Angular

Código en github

https://github.com/011011bjf/Guias_Practicas/tree/main/HTML/Guia_CRUD_Angular

1. Prerrequisitos y preparación del entorno

Antes de iniciar el desarrollo se verificó que el equipo cumpliera la línea base de versiones exigida, dado que Angular 22 no funciona con Node.js 20 ni con TypeScript anterior a la versión 6.

```
# 1. Comprobar la versión de Node (debe ser 22 o superior)
node --version
# v22.14.0

# 2. Instalar/actualizar la CLI de Angular de forma global
npm install -g @angular/cli@22

# 3. Confirmar la versión de la CLI y del framework
ng version
# Angular CLI: 22.0.0
# Angular: 22.0.0

# 4. Crear el proyecto (standalone y zoneless por defecto en v22)
ng new consumo-rest --style=css --ssr=false
cd consumo-rest

# 5. Levantar el servidor de desarrollo
ng serve -o
```

Listado 1. Verificación de versiones e inicialización del proyecto.

La opción -o de ng serve abre el navegador automáticamente en <http://localhost:4200>. El servidor de desarrollo recarga la página ante cada cambio guardado (live reload); no corresponde a un servidor de producción.

2. Arquitectura Fullstack de Referencia (Angular 22 + Spring Boot)

Con fines de contextualización académica, se documenta a continuación un escenario fullstack de referencia en el que el frontend Angular 22 descrito en este informe consume una API REST real, expuesta por un backend Spring Boot 4.1.0 conectado a una base de datos relacional PostgreSQL. Se detallan las versiones exactas del stack, la arquitectura en capas, la estructura del workspace y los comandos de operación del proyecto completo.

2.1 Stack tecnológico completo

La siguiente tabla resume los componentes, versiones exactas y el rol de cada tecnología empleada en ambos extremos de la solución.

Componente	Tecnología / Librería	Versión	Rol / Descripción
Backend Framework	Spring Boot	4.1.0	Framework principal de API REST (vía spring-boot-starter-parent)
Lenguaje Backend	Java	25	Versión de Java configurada en pom.xml (<java.version>25</java.version>)
Gestor de Dependencias	Apache Maven	Wrapper (mvnw)	Automatización de compilación y empaquetado

ORM / Persistencia	Spring Data JPA / Hibernate	4.1.0 (inferida)	Mapeo objeto-relacional y acceso a base de datos
Driver DB	PostgreSQL JDBC Driver	Runtime de Spring Boot	Conectividad JDBC con PostgreSQL (org.postgresql)
Herramientas Java	Lombok	optional / test	Generación automática de getters, setters, constructores y builders
Frontend Framework	Angular (Core & Common)	^22.0.0	Framework SPA moderno con componentes standalone
Frontend CLI & Build	@angular/cli / @angular/build	^22.0.1	Herramientas de compilación y servidor de desarrollo
Lenguaje Frontend	TypeScript	~6.0.2	Superconjunto tipado para JavaScript
Programación Reactiva	RxJS	~7.8.0	Manejo de flujos asíncronos y peticiones HTTP
Gestor de Paquetes	npm	11.13.0	Gestor de dependencias de Node.js (packageManager)

Tabla 1. Versiones exactas del stack tecnológico del proyecto fullstack de referencia.

2.2 Arquitectura en capas y flujo de datos

La solución sigue una arquitectura en tres capas físicas. En el frontend, los componentes `ListaProductosComponent` y `NuevoProductoComponent` inyectan un `ProductoService` que centraliza las llamadas HTTP mediante `HttpClient` y `RxJS`. En el backend, `ProductoController` expone los endpoints REST y delega en `ProductoService` (`@Service`), que a su vez utiliza `ProductoRepository` (`JpaRepository`) para acceder a la entidad `Producto` (`@Entity`) mapeada por Hibernate hacia PostgreSQL.

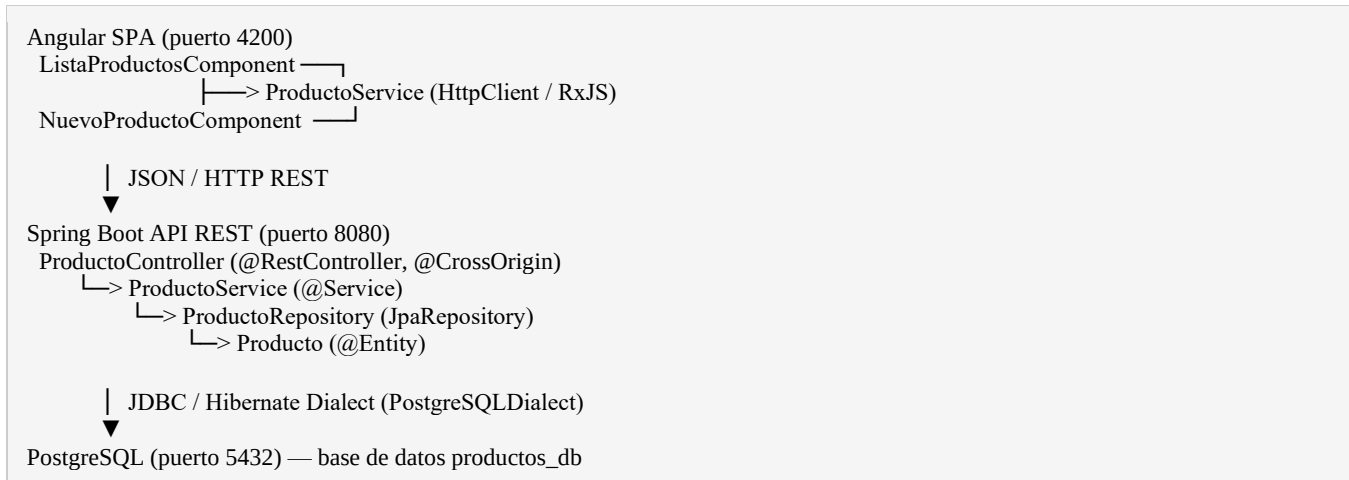
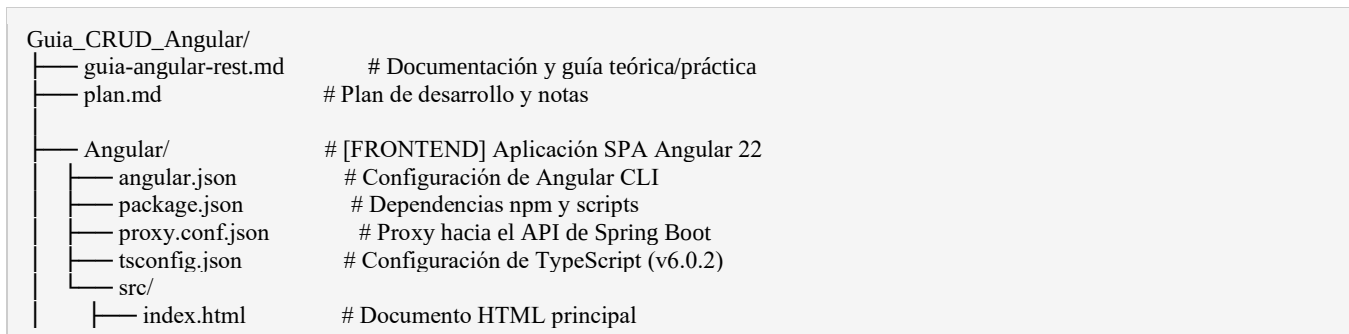


Figura 1. Diagrama de arquitectura y flujo de datos del sistema fullstack.

2.3 Estructura del workspace del proyecto

El workspace se organiza en dos directorios raíz independientes, Angular y Spring, junto con documentación de guía en la raíz del espacio de trabajo, tal como se muestra a continuación.



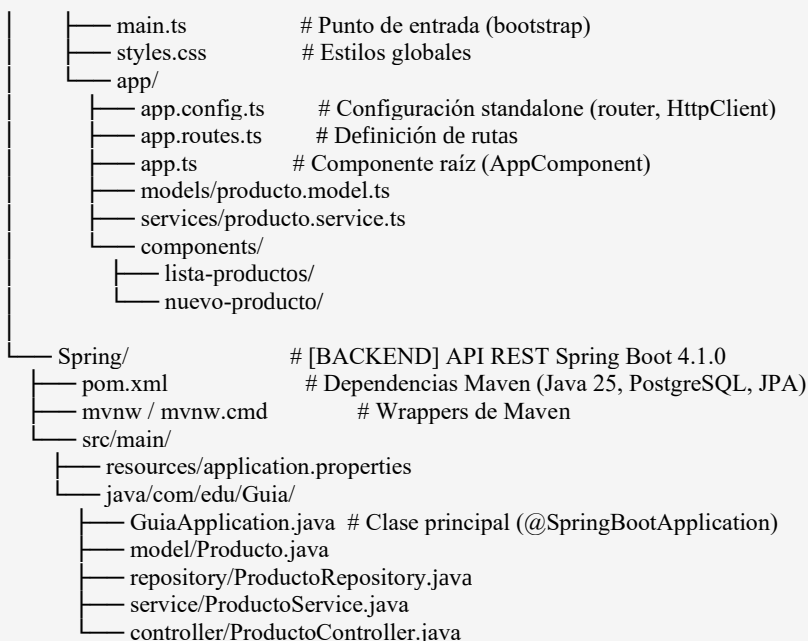


Figura 2. Árbol de directorios del workspace fullstack.

2.4 Descripción de componentes clave

Backend (Spring Boot):

- **application.properties:** configura el puerto 8080 y la conexión JDBC a jdbc:postgresql://localhost:5432/productos_db con el dialecto PostgreSQLDialect.
- **Producto.java:** entidad JPA que representa la tabla de base de datos, con anotaciones de persistencia de Spring Data JPA.
- **ProductoRepository.java:** interfaz que hereda de JpaRepository<Producto, Long>, obteniendo automáticamente findAll(), findById(), save() y deleteById().
- **ProductoService.java:** encapsula la lógica de negocio y las transacciones antes de llegar al controlador.
- **ProductoController.java:** expone las rutas RESTful (GET, POST, DELETE /api/productos) consumidas por Angular.

Frontend (Angular standalone):

- **producto.model.ts:** define la estructura tipada que coincide con los campos devueltos por la API.
- **producto.service.ts:** utiliza HttpClient y Observable (RxJS 7.8) para comunicarse con el backend, apoyándose en proxy.conf.json para evitar CORS en desarrollo.
- **lista-productos.component.ts:** obtiene la lista de productos en ngOnInit y gestiona la visualización y eliminación de registros.
- **nuevo-producto.component.ts:** gestiona el formulario de captura de datos y envía solicitudes POST de creación al servidor.

2.5 Comandos de operación y compilación

Backend (directorio Spring):

```

# Levantar el servidor en el puerto 8080 en modo desarrollo
.mvnw.cmd spring-boot:run

# Compilar y generar el artefacto .jar (sin ejecutar tests)
.mvnw.cmd clean package -DskipTests
  
```

Listado 11. Comandos de operación del backend Spring Boot.

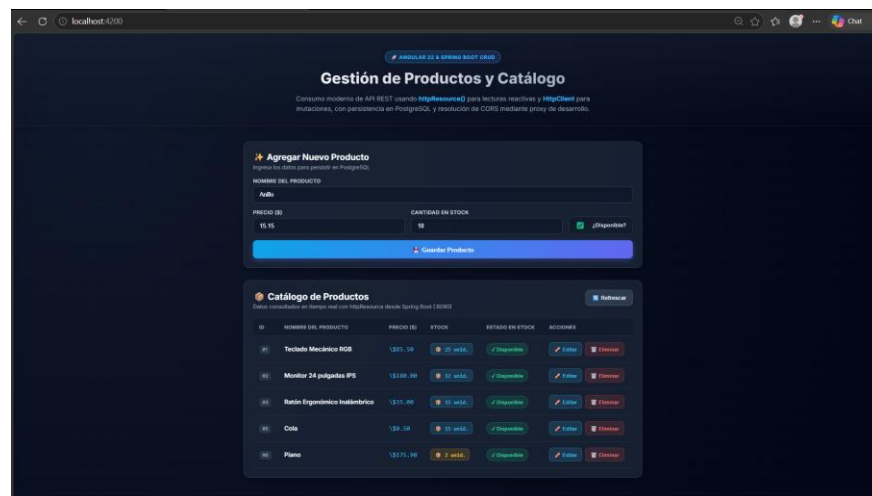
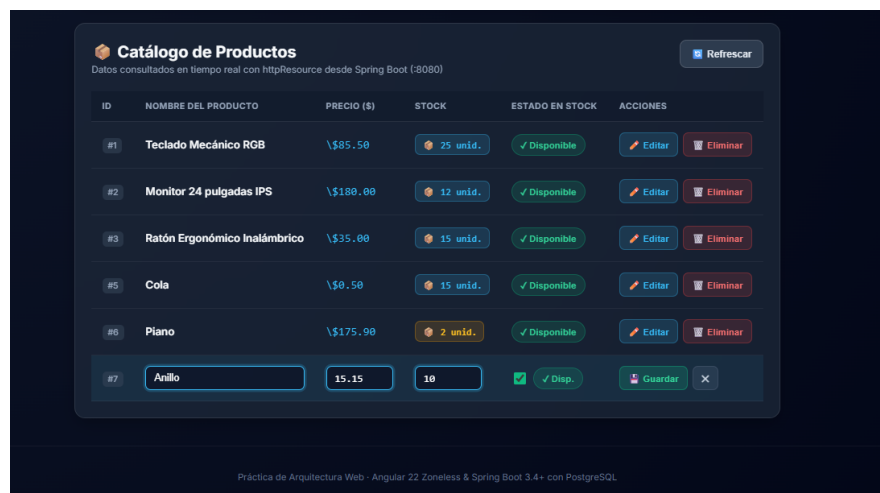
Frontend (directorio Angular):

```
# Instalar dependencias exactas de Node/Angular
npm install

# Iniciar servidor de desarrollo en http://localhost:4200 (con proxy)
npm start
# o directamente:
ng serve --proxy-config proxy.conf.json
```

Listado 12. Comandos de operación del frontend Angular.

3. Ejecución del programa

*Imagen 10 Evidencia de ejecución del programa en Angular y Spring boot**Imagen 11 Evidencia de Crud :Actualización*

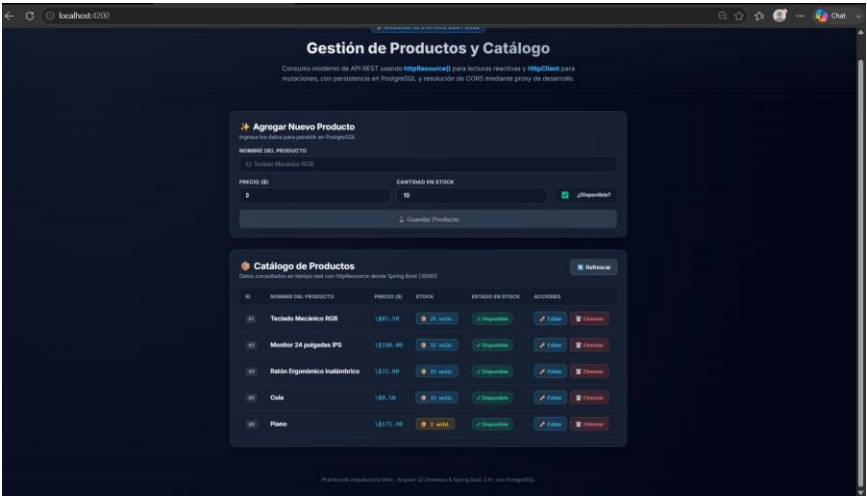


Imagen 12 Evidencia de Crud :Eliminación